

Programmation Orientée Objet en JAVA

Abdelwahab Naji

Table des matières

1	Introduction à la Programmation Orientée Objet (POO)	5
2	Concept de class & objet	7
2.1	Définitions & Exemple de mise en oeuvre	7
2.2	De la modélisation à l'implémentation	7
2.3	Création d'une classe vide	9
2.4	Les attributs	9
2.4.1	les attributs membres	9
2.4.2	Les attributs static	12
2.5	Les méthodes	14
2.5.1	Méthodes membres	14
2.5.2	Méthodes static	16
2.5.3	Conclusion	18
2.6	Utilisation avancée des éléments membres et static d'une classe	19
2.7	Exercices d'application : Méthodes membres et méthodes static	21
2.8	La surcharge (Overloading)	25
2.8.1	Surcharge de constructeur	25
2.8.2	Surcharge de méthodes	27
2.8.3	Exemple complet de surcharge	27
2.9	Exercices	30
3	Encapsulation en Programmation Orientée Objet (POO)	33
3.1	Mise en oeuvre de l'encapsulation	33
3.2	Rôle de l'encapsulation	35
3.3	Encapsulation & Getters et Setters en Java	36
3.3.1	Pourquoi utiliser les getters et setters ?	36
3.3.2	Application sur la classe <code>Produit</code>	37
3.3.3	Exemple d'utilisation dans le <code>main()</code> de la classe <code>Program</code>	37
3.3.4	Analyse du code source	38
3.3.5	La méthode <code>toString()</code>	38
3.3.6	Exemple de mise en œuvre : intégration de <code>toString()</code> dans la classe <code>Produit</code>	39
3.3.7	Exemple d'utilisation dans la classe <code>Program</code>	40
3.4	Exercices	40
4	Introduction aux Collections en Java	45
4.1	Présentation générale des collections	45
4.2	List et son implémentation <code>ArrayList</code>	45
4.3	Exemple avec des entiers	46
4.4	Exemple avec des objets <code>Produit</code>	47
4.5	Classe utilitaire <code>Collections</code>	48
4.6	Exercices	51

5	Conception et implémentation avancées : les relations entre les classes	53
5.1	Relation OneToOne	53
5.1.1	Description générale	53
5.1.2	Exemple : Classe Cercle et Point	53
5.1.3	Analyse	54
5.1.4	Code source pour tester et utiliser les éléments des classes précédentes	55
5.1.5	Analyse du programme	55
5.2	Relation ManyToOne	55
5.2.1	Description générale	55
5.2.2	Exemple : Ticket de café et Articles	55
5.2.3	Analyse	57
5.2.4	Code source pour tester et utiliser les éléments des classes précédentes	57
5.2.5	Analyse du programme	58
5.3	Exercices	58

Chapitre 1

Introduction à la Programmation Orientée Objet (POO)

La programmation orientée objet (POO) est une approche de développement qui permet de modéliser le monde réel sous forme d'objets. Chaque objet regroupe à la fois des données (appelées attributs) et des comportements (appelés méthodes). L'objectif principal de la POO est de rendre le code plus structuré, réutilisable et facile à maintenir. Dans ce document, les concepts suivants seront abordés :

Classe et Objet :

Le concept classe-objet constitue un principe fondamental de la POO, dont l'objectif est de définir des classes puis d'en créer des objets utilisables dans le programme.

La classe constitue le plan de conception d'un objet : elle décrit ses caractéristiques et ses actions possibles. L'objet, quant à lui, représente une instance concrète de cette classe, créée au moment de l'exécution du programme.

Ensemble, ces deux notions illustrent le passage de l'abstraction à la concrétisation : la classe fournit la description générale, tandis que l'objet en est la réalisation effective dans le programme.

Encapsulation :

L'encapsulation est un principe fondamental qui vise à protéger les données internes d'un objet contre tout accès direct depuis une méthode externe ou contre toute modification des données non contrôlée. Au lieu de manipuler directement les attributs d'un objet, on passe par des méthodes d'accès (appelées getters et setters) qui assurent un contrôle et une validation avant toute modification.

Ce mécanisme permet d'assurer la sécurité, la cohérence et la fiabilité du programme, tout en favorisant une meilleure isolation et centralisation du code et ainsi assurer une maintenance facile.

Importance de ces concepts :

Les notions de classe, objet et encapsulation constituent la base sur laquelle reposent les autres mécanismes de la POO. Elles préparent la compréhension de concepts plus avancés tels que :

L'héritage, qui favorise la réutilisation et l'extension du code. Le polymorphisme, qui permet d'adapter le comportement des objets selon le contexte.

Ce chapitre introductif a pour but de présenter la logique générale de la POO sans entrer dans les détails techniques. Les chapitres suivants approfondiront chacun de ces points :

Chapitre 2 – Classe et Objet : structure, instanciation, méthodes et constructeurs.

Chapitre 3 – Encapsulation : visibilité, getters, setters et bonnes pratiques.

Chapitre 2

Concept de class & objet

2.1 Définitions & Exemple de mise en oeuvre

Classe

Une classe est un modèle qui sert de plan de construction pour les objets. Elle décrit, de manière générale, les caractéristiques (attributs) et les comportements (méthodes) que partageront tous les objets du même type.

Autrement dit, la classe définit les propriétés et les actions communes à un ensemble d'objets, constituant ainsi une abstraction d'un concept réel. Par exemple :

- une classe **Voiture** (dans une application de jeu de courses) qui définit les caractéristiques (marque, capacité, couleur...) et les comportements (démarrer, accélérer, arrêter ...)
- une classe **Produit** (dans application de gestion de magasin) qui décrit les attributs d'un article (code, nom, quantité, prix) ainsi que des comportements tels que (mettre à jour la quantité, calculer le prix TTC, appliquer une remise...);
- une classe **Point** (dans une application de dessin) qui représente une position dans un plan (x, y) et propose des opérations comme (déplacer le point, calculer la distance à un autre point...);
- une classe **Salarie** (dans une application de gestion de ressources humaines) qui regroupe des informations telles que le matricule, le nom, le prénom et le salaire, et inclut des comportements comme (calculer le salaire annuel, appliquer une augmentation, afficher les informations du salarié...).

2.2 De la modélisation à l'implémentation

Dans une démarche de développement orienté objet, la modélisation UML, et en particulier le diagramme de classes, joue un rôle fondamental.

Le concept de classe occupe ainsi une place essentielle : il définit l'identité et le comportement des objets utilisés dans l'application. La figure 2.1, qui présente la notation UML d'une classe, offre une représentation claire de cette structure. Elle montre comment une classe est représentée graphiquement, en mettant en évidence son nom, ses attributs et ses méthodes.

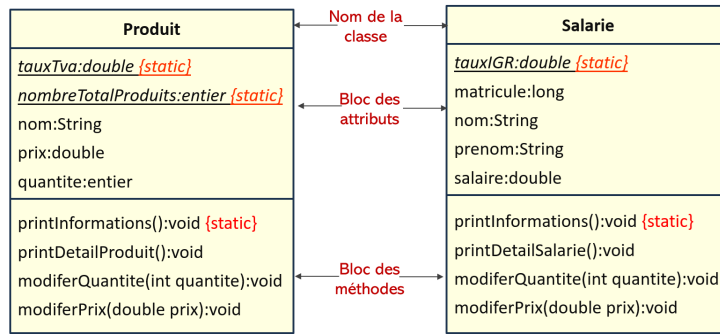


FIGURE 2.1 – Notation UML de la classe Produit et de la classe Salarie

Le diagramme de classes permet de représenter visuellement les éléments essentiels d'un système : les classes, leurs attributs, leurs méthodes, ainsi que les relations qui les lient (association, agrégation, composition, héritage...). Avant même d'écrire une seule ligne de code, ce diagramme offre une vision structurée et cohérente du futur programme. le diagramme ci dessous montre les classes à utiliser pour la gestion des comptes bancaires ainsi que les relations entre elles.

NB. la modélisation UML n'est pas traitée dans ce document. On se limite juste à utiliser des diagrammes de classes simples.

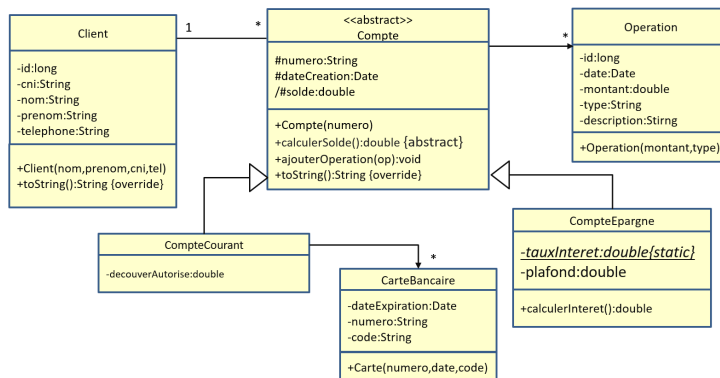


FIGURE 2.2 – Diagramme de classes- gestion des comptes bancaires

Grâce au diagramme de classes, le développeur peut analyser les responsabilités de chaque classe, identifier les dépendances, vérifier la cohérence du modèle et anticiper d'éventuelles améliorations ou simplifications.

Ainsi, la transition du diagramme de classes UML vers le code Java n'est pas un simple passage technique : c'est un processus de traduction logique, où chaque élément modélisé trouve sa représentation concrète dans le programme. Comprendre ce lien renforce la qualité du code, facilite la maintenance et garantit que l'implémentation reflète fidèlement la conception initiale.

Il est tout à fait possible d'écrire un code Java sans se référer à un diagramme de classes. Cependant, le diagramme de classes en notation UML constitue un outil précieux pour simplifier la conception et déterminer clairement les éléments à coder. Il permet de visualiser les classes, leurs attributs, leurs méthodes et les relations entre elles avant même de commencer l'implémentation. Pour cette raison, nous allons adopter une approche guidée par le diagramme de classes afin de traduire efficacement la conception en code Java.

Pour maîtriser efficacement le codage en Java, il est recommandé de procéder étape par étape. Plutôt que d'écrire l'ensemble du programme d'un seul coup, il est préférable de construire progressivement chaque classe. On commence par créer une classe vide, puis on y ajoute successivement les différents éléments : attributs, constructeurs, méthodes, et éventuellement des surcharges ou des modifications de visibilité. À chaque étape, il est important d'effectuer des tests afin de vérifier le fonctionnement des éléments ajoutés et de mieux comprendre leur utilisation. Cette approche permet également de mettre en pratique les concepts essentiels de la programmation orientée objet, tels que l'encapsulation et les relations entre classes, tout en garantissant un code clair et structuré.

2.3 Création d'une classe vide

En Java, la première étape consiste à déclarer une classe vide, qui servira de structure de base pour y ajouter les attributs et les méthodes. Avec ce bout de code source, nous avons créé un nouveau type que nous pouvons utiliser dans la suite. Par exemple, on peut mettre dans la méthode main de la classe Program l'instruction suivante : `Produit produit`.

```
1 class Produit {  
2     // Classe vide pour le moment  
3 }
```

dans la suite, cette classe sera mis à jour au fur et mesure pour contenir autres éléments.

2.4 Les attributs

Une fois la classe définie, on peut y intégrer les attributs qui représentent les caractéristiques de l'objet.

En **programmation orientée objet (POO)**, une **classe** représente à la fois des **données** et des **comportements**. Les données sont décrites à l'aide de **variables** déclarées à l'intérieur de la classe : on les appelle les **attributs**. Chaque attribut permet de définir une caractéristique de l'objet : par exemple, le nom, le prix ou la quantité d'un produit, les coordonnées d'un point, ou encore le salaire d'un employé.

Les attributs jouent donc un rôle central dans la définition de l'état d'un objet. Ils stockent des informations spécifiques qui peuvent être modifiées, lues ou exploitées par les **méthodes** de la classe. En Java, chaque objet créé à partir d'une même classe possède sa propre copie de ces attributs, ce qui garantit que chaque instance conserve son état propre et indépendant des autres.

On distingue deux grandes catégories d'attributs :

- Les **attributs membres (ou d'instance)**
- Les **attributs static (ou de classe)**

2.4.1 les attributs membres

- ils appartiennent à chaque objet créé à partir de la classe. Leur valeur peut donc varier d'un objet à un autre.

- Ainsi, les attributs membres décrivent l'état individuel de chaque objet

Le code source ci dessous, représente le code JAVA qui définit une classe Produit avec des attributs membres. Cette classe est définie par :

- l'attribut code : un entier qui représente le code d'un produit
- l'attribut nom : une chaîne de caractère (String) qui représente le nom d'un produit
- l'attribut qte : un entier qui représente la quantité d'un produit
- l'attribut prix : un double qui représente le prix d'un produit
- un constructeur : il s'agit d'une méthode spéciale ayant le même nom que la classe et sans type de retour. pour la classe Produit, le constructeur sans paramètres est Produit().... Dans la même classe, on peut trouver autres constructeurs avec paramètres. cf. section de la surcharge des méthodes et constructeurs.

si le constructeur contient un type de retour (void par exemple), il ne peut pas représenter le rôle d'un constructeur mais devient une méthode normale.

```

1 class Produit {
2     int code;
3     String nom;
4     int qte;
5     double prix;
6     Produit() {
7         System.out.println("Créer d'un produit...");
8     }
9 }

```

Code 2.1 – Code source de la classe Produit

Analyse du code source

- Ligne 1 : Elle contient le nom de classe
- Ligne 2 à 5 : définition des attributs de la classe
Les attributs sont : code, nom, qte, prix. Ils représentent les caractéristiques d'un produit.
- Ligne 6 à 8 : définition du constructeur exécutée automatiquement à la création de chaque objet. Il affiche un message confirmant la création de l'objet.
- Le constructeur est une méthode spéciale : Il porte le même nom que la classe et sert à initialiser les attributs d'un objet.
- Le constructeur par défaut ne contient pas de paramètres Il existe implicitement, mais on peut le personnaliser pour effectuer des traitements
- Un constructeur n'a pas de type de retour : Contrairement aux méthodes classiques (à voir par la suite), il ne retourne aucune valeur explicite.

Si un type de retour est défini, il ne s'agit plus d'un constructeur mais d'une méthode ordinaire.

```

1 class Produit{
2     void Produit(){} //une méthode
3 }

```

Code 2.2 – Méthode ayant le nom de la classe

En résumé : cette classe ne contient pas encore de méthodes, elle contient uniquement les attributs (la structure) d'un produit et le code source du constructeur par défaut.

Objet

- Un objet est une instance d'une classe : Il représente une entité concrète créée à partir du modèle défini par la classe.
- On crée un objet en utilisant le mot-clé `new` : Ce mot-clé est suivi d'un constructeur, qui initialise l'objet.
- Une fois créé, un objet occupe une zone mémoire : Cette zone mémoire contient les valeurs de ses attributs et est identifiée par une référence (adresse mémoire).

NB. en général, les objets sont créés dans autres classes et leurs cycles de vie dépend de leurs utilisations.

Le code suivant montre comment :

1. Créer des objets de la classe `Produit`
2. Afficher la valeur de la référence (adresse mémoire)
3. Accéder à leurs attributs pour les modifier
4. Accéder à leurs attributs pour les afficher

```
1  class Program{
2
3  public static void main(String T[]){
4      Product p1=new Product();
5      Product p2=new Product();
6      new Product(); //objet sans référence, il ne peut jamais être
                       //utilisé dans la suite, sauf s'il passé en paramètres d'une
                       //méthode
7      System.out.println("@memoire de p1:"+p1); //p1 est équivalent
                       //à Produit@10F10 de la figure 2.2
8
9      System.out.println("@memoire de p2:"+p2); //p2 est équivalent
                       //à Produit@20F20 de la figure 2.2
10
11     //accès à la zone de p1 pour modification
12     p1.code=123;
13     p1.nom="S23";
14     p1.quantite=10;
15     p1.prix=3000.0;
16     //accès à la zone de p2 pour modification
17     p2.code=456;
18     p2.nom="S24";
19     p2.quantite=15;
20     p2.prix=5000.0;
21     //afficher le détail de p1
22     System.out.println("nom:"+p1.nom); //accéder à l'attribut nom
                       //pour afficher sa valeur
23     System.out.println("qte:"+p1.qte);
24     System.out.println("prix:"+p1.prix);
25     System.out.println("Total:"+(p1.qte*p1.prix));
26 }
27 }
```

Code 2.3 – Créer et manipuler des objets

- Ligne 4 : création d'un objet p1 de type Produit.
- Ligne 5 : création d'un objet p2 de type Produit.
- Ligne 6 : création d'un objet anonyme (pas de référence) de type Produit. (cf Etat initial de la Figure 2.2) La création d'un objet se fait obligatoirement à l'aide du mot clé new. le mot clé new doit être suivi par le constructeur Produit(). le résultat du new Produit(); est une adresse mémoire qui est stocké dans la référence p1.
- ligne 16 à 15 : modifier les valeurs des attributs spécifiques à p1. (cf Etat final de p1 de la Figure 2.2)
- ligne 17 à 20 : modifier les valeurs des attributs spécifiques à p2. (cf Etat final de p2 de la Figure 2.2)
- ligne 22 à 25 : afficher les valeurs des attributs de p1

Pour bien comprendre le fonctionnement interne d'un programme, il est très utile de schématiser la mémoire occupée par chaque objet pendant l'exécution. Ce type de représentation permet de visualiser clairement où et comment chaque attribut est stocké, quelles références pointent vers quels objets, et comment les modifications affectent l'état interne des objets. Schématiser la mémoire facilite ainsi la compréhension du comportement du code derrière les abstractions de la programmation orientée objet et constitue un outil pédagogique précieux pour analyser et déboguer un programme.

La figure ci dessous montre le résultat de l'exécution du code 2.3.

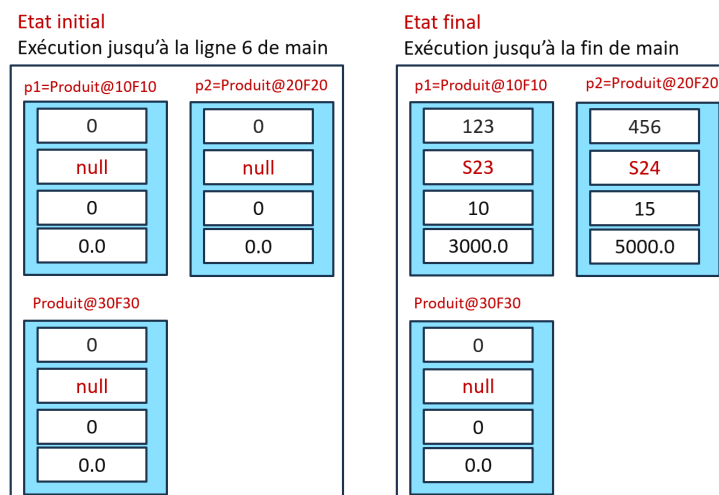


FIGURE 2.3 – Zones mémoires allouées

Chaque objet possède sa propre zone mémoire (illustrée en bleu), dans laquelle sont stockés tous les attributs définis par la classe.

Lors de la création d'un objet, ces attributs reçoivent des valeurs par défaut :

- les types numériques sont initialisés à 0,
- les attributs de type String sont initialisés à null.

Remarque : Une variable de type String est en réalité une référence vers un objet. Ainsi, tout attribut déclarant un type objet (comme String ou toute classe personnalisée) est automatiquement initialisé à null tant qu'aucune instance ne lui est affectée.

2.4.2 Les attributs static

- Ils sont partagés par l'ensemble des objets d'une même classe. Une seule copie existe en mémoire, commune à toutes les instances.

- Ils permettent de représenter des informations globales
- le code ci dessous intègre deux attributs static :
- `tauxTVA` : la teux de tva appliqué à l'ensemble de produits
- `nombreProduits` : le nombre total des produits créés lors de l'exécution de l'appli-
cation

```

1  class Produit {
2      int code;
3      String nom;
4      int qte;
5      double prix;
6      static double tauxTVA = 0.2;
7      static int nombreProduits = 0;
8
9      Produit() {
10         nombreProduits++;
11     }
12 }

```

Code 2.4 – Classe Produit avec attributs static

La figure ci-dessous illustre les zones mémoire allouées lors de l'exécution du code 2.4, ainsi que leur contenu.

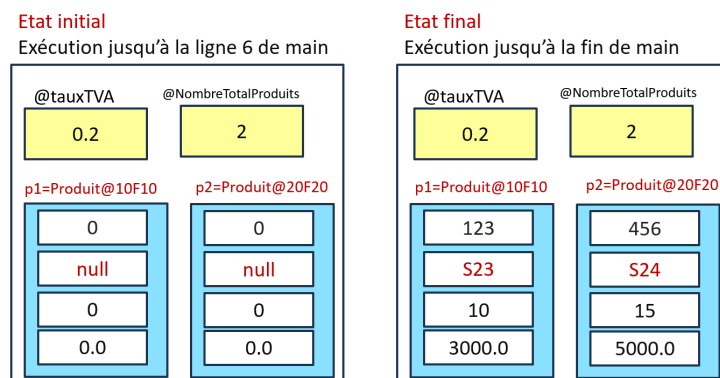


FIGURE 2.4 – Zones mémoires allouées

Les attributs d'instance (`code`, `nom`, `prix`, `quantité`) sont stockés dans des zones bleues, propres à chaque objet : chaque objet dispose ainsi de sa propre copie de ces valeurs.

À l'inverse, l'attribut statique `tauxTVA`, dont la valeur est 0.2, est partagé par tous les objets de la classe `Produit`. Il est représenté dans la zone jaune, séparée des zones dédiées aux objets, car sa valeur est unique au niveau de la classe et non dupliquée pour chaque instance.

Analyse du code source

- ligne 2 à 5 : création des attributs membres
- ligne 6 à 7 : création des attributs static
- ligne 10 : incrémentation du nombre d'objets créé

Accès aux attributs static

Les attributs static s'utilisent sans créer d'objet en passant par le nom de la classe. le code ci dessous montre un exmple.

```

1 System.out.println("Taux TVA: " + Produit.tauxTVA);
2 System.out.println("Nombre total de produits: " + Produit.
    nombreProduits);

```

Code 2.5 – Accès aux attributs static

En résumé, les attributs static permettent de gérer des données communes à tous les objets d'une même classe. Ils sont souvent utilisés pour représenter des constantes (comme un taux de TVA) ou pour suivre un état global (comme le nombre d'objets créés).

2.5 Les méthodes

Les comportements d'une classe sont définis à travers ses méthodes, qui permettent de manipuler les attributs et de préciser les actions qu'un objet peut accomplir. Contrairement aux structures en C, limitées aux données, une classe en Java peut intégrer des fonctions. En programmation orientée objet, ces fonctions sont appelées méthodes, et elles décrivent les comportements associés soit aux objets, soit à la classe elle-même.

Catégories de méthodes

Il existe deux catégories principales de méthodes :

- **Méthodes membres (ou d'instance)**
 - Liées à un objet spécifique : son exécution concerne un objet (donc une zone mémoire).
 - Peuvent manipuler les attributs d'instance et les attributs `static`.
 - Ne peuvent être appelées qu'à travers un objet : `objet.methodeMembre()`;
- **Méthodes static (ou de classe)**
 - Concernent la classe elle-même et non les objets créés à partir de cette classe.
 - Peuvent manipuler uniquement les attributs `static`.
 - Sont accessibles via le nom de la classe : `NomClasse.methodeStatic()`;
 - Peuvent aussi être appelées via un objet (`objet.methodeStatic()`), mais cela est déconseillé.

2.5.1 Méthodes membres

le code source ci dessous représente la classe `Produit`.

```

1 public class Produit {
2     int code;
3     String nom; //attribut membre
4     int quantite;
5     double prix;
6     // ===== Méthode membre =====
7     void printDetailProduit() {
8         System.out.println("@mémoire: " + this);
9         System.out.println("Code: " + this.code);
10        System.out.println("Nom: " + nom);
11        System.out.println("Quantité: " + quantite);

```

```

12         System.out.println("Prix: " + prix + " DH");
13     }
14 }

```

Code 2.6 – Classe Produit avec méthode membre & mot clé this

le code source ci dessous représente la classe Program qui utilise la classe Produit.

```

1  class Program {
2      public static void main(String[] args) {
3          Produit p1 = new Produit();
4          p1.code = 111;p1.nom = "PC";p1.quantite = 5;p1.prix =
5              6780;
6          p1.printDetailProduit();
7          Produit p2 = new Produit();
8          p2.code = 222;p2.nom = "Imprimante";p2.quantite = 10;p2.
          prix = 2500;
9          p2.printDetailProduit();
10     }
11 }

```

Code 2.7 – Utilisation d'une méthode membre

après exécution de la classe Program, le résultat montré dans la figure ci dessous est obtenu.

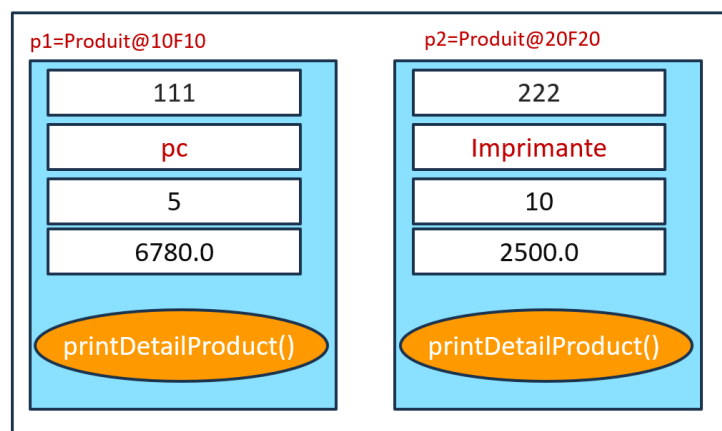


FIGURE 2.5 – Zones mémoires allouées avec méthodes

Cette figure illustre la répartition en mémoire lors de la création de deux objets à partir de la classe `Produit`. Chaque zone bleue représente l'espace mémoire propre à un objet, contenant ses attributs d'instance (code, nom, prix, quantité). La classe `Produit` fournit également la méthode `printDetailProduit()`, représentée dans chaque zone par la zone en orange. Cela montre qu'une même méthode définie dans la classe peut être appelée indépendamment par chaque objet, tout en utilisant les valeurs spécifiques stockées dans sa propre zone mémoire.

Analyse du code source du code 2.6

Dans la classe `Produit`

- Ligne 2 à 5 : ajouter des attributs membres à la classe `Produit`
- Ligne 7 à 13 : le corps de la méthode `printDetailProduit`

- ligne 7 : définition de la signature de la méthode membre printDetailProduit
- Ligne 8 : afficher la zone mémoire créée par le constructeur
- Ligne 9 à 12 : afficher le contenu des attributs de l'objet qui va appeler la méthode (ligne 6 et 9 du code 2.7)
 - => pour l'appel p1.printDetailProduit() de la ligne 6 de la classe Program, @memoire de p1 est affiché
 - => pour l'appel p2.printDetailProduit() de la ligne 9 de la classe Program, @memoire de p2 est affiché

Analyse du code source du code 2.7

Dans la classe Program

- Ligne 4 à 5 : création et initialisation de l'objet p1
- Ligne 6 : appel de la méthode printDetailProduit() depuis p1
- Ligne 7 à 8 : création et initialisation de l'objet p1
- Ligne 9 : appel de la méthode printDetailProduit() depuis p2

2.5.2 Méthodes static

Ajouter une méthode static à la classe Produit

il est important à préciser que pendant l'exécution d'une application java, l'appel et le comportement (exécution) d'une méthode static ne dépend pas d'un objet spécifique (autrement dit d'une zone mémoire spécifique), mais de la zone mémoire qui concerne toute la classe. On dit aussi que son comportement est partagé entre tous les objets de la classe. Le code source java suivant permet d'ajouter une méthode de signature :

static void printInfosGeneral()

à la classe Produit.

le code source ci dessous, montrer comment ajouter la méthode static printInfosGeneral() à la classe Produit

```

1 public class Produit {
2     int code;
3     String nom; //attribut membre
4     int quantite;
5     double prix;
6     static double tauxTva = 0.20; //attribut static
7     static int nombreProduits = 0;
8     // ===== Méthode static =====
9     static void printInfosGeneral() {
10         System.out.println("Taux TVA: " + tauxTva);
11         System.out.println("nombre Produits: " + nombreProduits);
12         // System.out.println("@memoire: " + this); //pas de this
13         // System.out.println("Prix: " + prix + " DH"); // pas de
            prix
14     }
15     // ===== Méthode membre =====
16     void printDetailProduit() {
17         System.out.println("@mémoire: " + this);
18         System.out.println("Code: " + this.code);
19         System.out.println("Nom: " + nom);
20         System.out.println("Quantité: " + quantite);

```



```
21         System.out.println("Prix: " + prix + " DH");
22     }
23 }
```

Code 2.8 – Classe Produit avec méthode static

le code suivant illustre comment utiliser la méthode static `printInfosGeneral()` de différentes manières.

```
1
2 class Program {
3     public static void main(String[] args) {
4         Produit.tauxTva=0.25;
5         Produit.printInfosGeneral();
6         Produit p1 = new Produit();
7         p1.code = 111;p1.nom = "PC";p1.quantite = 5;p1.prix =
            6780;
8         p1.printDetailProduit();
9         //Produit.quantite=55; appel incorrect
10        //Produit.printDetailProduit(); appel incorrect
11        Produit p2 = new Produit();
12        p2.code = 222;p2.nom = "Imprimante";p2.quantite = 10;p2.
            prix = 2500;
13        p2.printDetailProduit();
14        p2.printConfig();
15        p1.tauxTva=0.30;
16        p1.printConfig();
17        p2.printInfosGeneral();
18        Produit.printInfosGeneral();
19    }
20 }
```

Code 2.9 – Classe Produit avec méthode static

la figure ci dessous montre les zones mémoires allouées ainsi que leurs contenu après exécution du code 2.9. notamment celle de la méthode static.

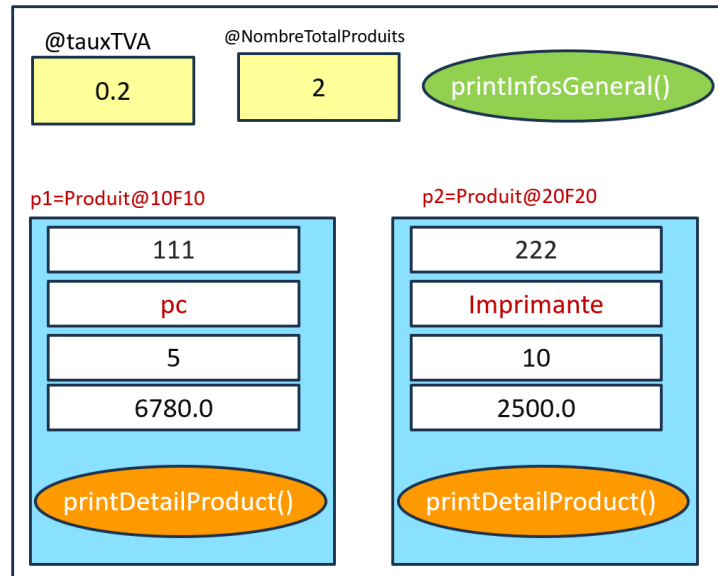


FIGURE 2.6 – Méthode static vs mémoire d'exécution

cette figure montre que l'exécution de `printInfosGeneral()`, située dans la zone verte, ne concerne pas une zone mémoire spécifique à un objet mais plutôt ça concerne toute la classe. Autrement dit, son comportement est partagé entre tous les objets. **Analyse du code source**

Dans la classe `Produit`

- Ligne 9 : définition de la signature de la méthode static `printInfosGeneral()`
- Ligne 10 à 14 : le corps de la méthode `printInfosGeneral()`
- Ligne 10 : accéder à l'attribut static `tauxTva`
- Ligne 11 : accéder à l'attribut static `nombreProduits`
- Ligne 12 : il n'y a pas de `this` au sein d'une méthode static. donc son utilisation au sein d'une méthode static donne lieu à une erreur de compilation.
- Ligne 13 : le prix est un attribut membre. donc son utilisation au sein d'une méthode static donne lieu à une erreur de compilation.
- Ligne 16 : définition de la signature de la méthode membre `printDetailProduit`
- Ligne 17 : afficher la mémoire qui concerne l'objet qui va appeler cette méthode

Dans la classe `Program`

- Ligne 4 : accéder à l'attribut static par le nom de la classe pour le modifier
- Ligne 5 : appeler la méthode static `printInfosGeneral()` par le nom de la classe
- Ligne 9-10 : on ne peut pas accéder à un élément membre par le biais du nom de la classe.
- Ligne 14 à 17 : un objet de `Produit` peut accéder aux éléments static.
- Ligne 18 : résultat de l'appel :
 Taux TVA : 0.3
 nombre Produits : 2

2.5.3 Conclusion

dans cette partie, nous avons présenté :

- comment créer et utiliser une méthode membre
- que signifie le mot-clé `this` et comment l'utiliser

- comment créer et utiliser une méthode static
- connaître la différence entre une méthode membre et une méthode static
- proposer un schéma de mémoires des éléments membres et static pendant l'exécution d'un programme java

2.6 Utilisation avancée des éléments membres et static d'une classe

L'objectif est de mettre à jour le code source de la classe produit en ajoutant les méthodes suivantes :

- `void modifierQuantite(int q)` : méthode membre pour modifier la quantité d'un produit.
- `void modifierPrix(double pr)` : méthode membre pour modifier le prix d'un produit.
- `double calculerTotalHT()` : méthode membre pour calculer le total hors taxe (`prix * quantite`).
- `double calculerTotalTTC()` : méthode membre pour calculer le total toutes taxes comprises (`totalHT + totalHT * tauxTva`).
- `printNombreTotalProduit()` : méthode static pour afficher le nombre total de produits créés.
- `void printDetailProduitV2(Produit p)` : méthode static pour afficher le détail d'un produit.
- question de réflexion : en comparant la méthode `printDetailProduitV2` et la méthode `printDetailProduit`, Pourquoi la méthode `printDetailProduitV2` doit recevoir en paramètres la référence du produit dont on veut afficher le détail

Code source qui intègre ces méthodes

```

1  class Produit {
2      int code;
3      String nom;
4      int quantite;
5      double prix;
6      static double tauxTva = 0.2;
7      static int nombreProduits = 0;
8
9
10     // ===== Méthodes membres =====
11     void printDetailProduit() {
12         System.out.println("Code: " + code);
13         System.out.println("Nom: " + nom);
14         System.out.println("Quantité: " + quantite);
15         System.out.println("Prix: " + prix + " DH");
16     }
17
18     void modifierQuantite(int quantite) {
19         this.quantite = quantite;
20     }

```

```

21
22     void modifierPrix(double prix) {
23         this.prix = prix;
24     }
25
26     double calculerTotalHT() {
27         return prix * quantite;
28     }
29
30     double calculerTotalTTC() {
31         double totalHT = calculerTotalHT();
32         return totalHT + totalHT * tauxTva;
33     }
34
35     // ===== Méthodes static =====
36     static void printNombreTotalProduit() {
37         System.out.println("Nombre total de produits créés: " +
38             nombreProduits);
39     }
40
41     static void printDetailProduitV2(Produit p) {
42         System.out.println("Code: " + p.code);
43         System.out.println("Nom: " + p.nom);
44         System.out.println("Quantité: " + p.quantite);
45         System.out.println("Prix: " + p.prix + " DH");
46     }

```

Code 2.10 – Classe Produit avec méthodes membres et static

Exemple d'utilisation

```

1  public class Program_Produit {
2      public static void main(String[] args) {
3          Produit p1 = new Produit(101, "Galaxy S24", 20, 4500);
4          Produit p2 = new Produit(102, "iPhone 16", 10, 8900);
5          p1.printDetailProduit();
6          System.out.println("Total TTC de p1: " + p1.
7              calculerTotalTTC());
8          p2.modifierPrix(9200);
9          p2.printDetailProduit();
10         // Appel de la méthode static
11         Produit.printDetailProduitV2(p1);
12         Produit.printNombreTotalProduit();
13     }
14 }

```

Code 2.11 – Exemple d'utilisation des méthodes

7. Conclusion

Les **méthodes** représentent le comportement d'une classe. Elles permettent d'interagir avec les données internes (attributs) et de réaliser des traitements. Les **méthodes static** servent aux traitements globaux liés à la classe, tandis que les **méthodes membres** gèrent le comportement spécifique de chaque objet.

2.7 Exercices d'application : Méthodes membres et méthodes static

Les exercices suivants visent à renforcer la compréhension des notions de **méthodes membres** et de **méthodes static** à travers deux classes simples : **Point** et **Salarie**. L'objectif est de distinguer les comportements propres à un objet et ceux partagés par l'ensemble de la classe.

Exercice 1 : Analyse de code source

L'objectif de cet exercice est d'avoir les compétences suivantes :

- Analyser un code source
- suivre une exécution pas à pas pour détecter les erreurs et les anomalies
- fournir le résultat d'un programme

Program 1

```

1  class Loo{
2  int x; int y; //ligne à ne pas modifier
3  void display(){
4  System.out.println(x+ " "+y);
5  }
6  }
7  public class Question1 {
8  static void doIt(Loo lo){
9  lo=new Loo();
10 lo.x=lo.x+5; lo.y=lo.y-5;
11 }
12 public static void main(String[] args) {
13 Loo lo=new Loo();
14 doIt(lo);
15 lo.display();
16 }
17 }

```

Code 2.12 – Program 1

Questions :

1. Si **Program1** ne contient pas d'erreur d'exécution :
 - Résultat affiché par le programme :

2. Si le programme contient des erreurs :

— Lignes contenant des erreurs :

— Corrections à effectuer :

— Résultat à afficher après correction :

Program 2

```
1  class Foo{
2  static int x;static int y; //ligne à ne pas modifier
3  void display(){
4  System.out.println(x+ " "+y);
5  }
6  Foo(){x=x+5;y=y+5;}
7  }
8  public class Question2 {
9  static void doIt(Foo fo){
10 fo=new Foo();
11 fo.x=fo.x+5; fo.y=fo.y-5;
12 }
13 public static void main(String[] args) {
14 Foo fo=new Foo();
15 doIt(fo);
16 fo.display();
17 }
18 }
```

Code 2.13 – Program 2

Questions :**1. Si Program2 ne contient pas d'erreur d'exécution :**

— Résultat affiché par le programme :

2. Si le programme contient des erreurs :

— Lignes contenant des erreurs :

— Corrections à effectuer :

— Résultat à afficher après correction :

Exercice 2 : Classe Point

On considère une classe `Point` définie par deux attributs : `x` et `y`.

Travail demandé :

1. Créer une méthode membre permettant d'afficher les coordonnées d'un point sous la forme `(x,y)`.
2. Créer une méthode membre `deplacer(int dx,int dy)` permettant de modifier les coordonnées du point selon deux valeurs fournies en paramètres.
3. Créer une méthode membre `getDistance(Point p)` permettant de calculer la distance entre le point courant et un autre point donné en paramètre.
4. Réfléchir : Peut-on appeler la méthode `deplacer()` sans objet ?
5. Est ce qu'on peut créer une méthode `static` ? qui permet de calculer et retourner la distance entre deux points. si Oui, donner son code source.
6. Créer une classe exécutable `Program` qui permet de tester les fonctionnalités précédentes.

Exercice 3 : Gestion des Salariés

Cet exercice vise à manipuler les membres d'instance, les membres `static`, ainsi que l'organisation en plusieurs classes.

1. Classe Salarie

Créer une classe **Salarie** contenant les attributs suivants :

- **matricule** (int)
- **nom** (String)
- **prenom** (String)
- **salaire** (double)
- **static int nombreSalaries** qui compte le nombre total d'objets créés

Travail demandé :

1. Créer une méthode membre **afficherInfos()** qui affiche les informations du salarié.
2. Créer une méthode membre **augmenterSalaire(double taux)** qui augmente le salaire selon un pourcentage.
3. Créer une méthode **static printNombreSalaries()** qui affiche le nombre total de salariés.
4. Répondre : Peut-on appeler **afficherInfos()** sans créer un objet ? Pourquoi **printNombreSalaries()** est-elle une méthode **static** ?
5. Ajouter une méthode **static afficherDetail(Salarie s)** qui affiche le détail d'un salarié passé en paramètre.

2. Classe SalarieManagement

1. reproduire les mêmes fonctionnalités de la classe **Salarie**, mais uniquement avec des méthodes **static**.
 - Méthode **static afficherInfos(Salarie s)**
 - Méthode **static augmenterSalaire(Salarie s, double taux)**
 - Méthode **static printNombreSalaries()**
2. pourquoi les méthodes précédentes doivent recevoir une référence de type **Salarie**

3. Classe Program

Créer une classe **Program** permettant de :

- instancier plusieurs objets **Salarie**
- tester les méthodes d'instance de **Salarie**
- tester les méthodes **static** de **SalarieManagement**
- créer une méthode **static afficherDetail(Salarie s)** pour afficher le détail d'un salarié

Objectif : Savoir distinguer les comportements dépendant d'un objet (méthodes membres) et ceux qui concernent l'ensemble de la classe (méthodes static).

2.8 La surcharge (Overloading)

La **surcharge** (*overloading*) est un concept fondamental de la **programmation orientée objet (POO)** qui consiste à définir plusieurs **méthodes** ou **constructeurs** portant le même nom mais ayant des **signatures différentes** — c'est-à-dire un nombre ou des types de paramètres différents.

Elle permet :

- d'améliorer la lisibilité du code ;
- d'augmenter la flexibilité dans l'appel des méthodes ;
- d'offrir plusieurs façons d'initialiser ou de modifier un objet selon les besoins.

Le concept de **surcharge** en programmation orientée objet englobe deux formes principales : la **surcharge des constructeurs** et la **surcharge des méthodes**.

2.8.1 Surcharge de constructeur

Le constructeur d'une classe est dit surchargé, si au moins il contient un constructeur avec au moins un paramètre.

Constructeur par défaut : Le constructeur par défaut n'a aucun paramètres. l'exemple ci dessous définit un constructeur sans paramètres.

```

1 Produit() {
2     System.out.println("Création d'un produit...");
3     this.quantite=1; //initialisation en dur.
4
5 }
```

Code 2.14 – Constructeur par défaut

Analyse du code du constructeur par défaut

- Il sert à créer un produit sans donner la possibilité de fournir des valeurs initiales depuis un code externe, mais on peut appliquer une initialisation en dur.
- tous les produits créés à l'aide de ce constructeur ont quantite=1 juste après leurs création.

Constructeur avec paramètres : appelé aussi **constructeur surchargé**, un constructeur avec paramètres reçoit au moins un paramètre. le code ci dessous définit un constructeur capable d'initialiser les attributs d'un objet de la classe Produit

```

1 Produit(int code, String nom, int qte, double prix) {
2     this.code = code;
3     this.nom = nom;
4     this.qte = qte;
5     this.prix = prix;
6 }
```

Code 2.15 – Constructeur avec paramètres

Analyse du code du constructeur avec paramètres

- Il permet d'initialiser directement les attributs lors de la création de l'objet.
- chaque produit créé avec ce constructeur a son propre état initial.

Exemple d'utilisation des deux constructeurs précédents :

```

1 Produit p1 = new Produit(); // constructeur par défaut
2 Produit p2 = new Produit(123, "Galaxy S44", 50, 4500.5); //
   constructeur surchargé

```

Remarque importante sur le constructeur par défaut**Remarque importante sur le constructeur par défaut**

En Java, il n'est pas nécessaire d'écrire explicitement un constructeur par défaut (c'est-à-dire un constructeur sans paramètres) sauf dans le cas suivant :

- la classe définit déjà un ou plusieurs constructeurs avec paramètres **et** on souhaite également pouvoir créer des objets sans fournir d'arguments.

Précisions :

- Si la classe ne possède **aucun** constructeur déclaré, Java génère automatiquement un constructeur par défaut.
- Si la classe possède au moins un constructeur avec paramètres, alors Java **ne** génère **pas** de constructeur par défaut

dans cet exemple, le constructeur par défaut n'est pas généré

Exemple :

```

1 public class Produit {
2     private int code;
3     private String nom;
4     private int quantite;
5     private double prix;
6
7     // Constructeur avec paramètres
8     public Produit(int code, String nom, int quantite, double
9         prix) {
10         this.code = code;
11         this.nom = nom;
12         this.quantite = quantite;
13         this.prix = prix;
14     }
15
16 }

```

Exemples d'utilisation :

```

1 Produit p1 = new Produit(101, "Stylo", 20, 3.5); // OK :
   constructeur paramétré
2 Produit p2 = new Produit(); // Erreur d'
   appel: le constructeur par défaut n'est pas défini

```

Dans cette situation :

- Java **ne génère plus** de constructeur par défaut ;
- la ligne suivante provoquera une erreur :

```

1 Produit p = new Produit(); // Erreur : aucun constructeur par d
   éfaut n'existe

```

Si l'on souhaite permettre la création d'un produit sans fournir de paramètres, il faut alors définir manuellement un constructeur par défaut. cad ajouter dans le code de la classe PProduit, le code source suivant :

```
1 public Produit() {
2     this.code = 0;
3     this.nom = "Inconnu";
4     this.quantite = 0;
5     this.prix = 0.0;
6 }
```

On peut alors l'utiliser dans la classe Program :

```
1 Produit p1 = new Produit(101, "Stylo", 20, 3.5); // constructeur
    avec paramètres
2 Produit p2 = new Produit(); // constructeur
    par défaut
```

2.8.2 Surcharge de méthodes

pour mettre en oeuvre la surcharge d'une méthode, nous allons mettre en place un exemple en utilisant la même classe Produit.

dans l'exemple ci dessous, nous allons définir une méthode modifier(int quantite) -version 1- permettant de modifier la quantité d'un produit, puis la redéfinir pour qu'elle soit capable de

- modifier le nom : **void modifier(String nom)** -version 2-
- modifier à la fois la quantité et le prix : **void modifier(int quantite, double prix)** -version 3-
- modifier à la fois la quantité, le prix et le nom : **void modifier(int quantite, double prix, String nom)** -version 4-

```
1 void modifier(String nom) { ... }
2 void modifier( int quantite, double prix) { ... }
3 void modifier(int quantite, double prix, String nom) { ... }
```

Code 2.16 – Méthodes surchargées

Exemple d'utilisation dans le code :

```
1 p1.modifier("Galaxy S224"); // appel de la version 1
2 p2.modifier("Galaxy S44", 105); // appel de la
    version 3
```

2.8.3 Exemple complet de surcharge

Pour illustrer la surcharge à la fois des **constructeurs** et des **méthodes**, considérons l'exemple suivant comportant deux classes : Produit et Program.

```
1 public class Produit {
2     int code;
3     String nom;
4     int quantite;
```

```
5     double prix;
6
7     // --- Constructeur par défaut ---
8     Produit() {
9         System.out.println("Création d'un produit par défaut...")
10        ;
11        this.quantite = 1; // Initialisation en dur
12    }
13
14    // --- Constructeur avec paramètres ---
15    Produit(int code, String nom, int quantite, double prix) {
16        this.code = code;
17        this.nom = nom;
18        this.quantite = quantite;
19        this.prix = prix;
20        System.out.println("Création d'un produit avec paramètres
21        ...");
22    }
23
24    // --- Méthodes surchargées ---
25
26    // Version 1 : modifier la quantité
27    void modifier(int quantite) {
28        this.quantite = quantite;
29    }
30
31    // Version 2 : modifier le nom
32    void modifier(String nom) {
33        this.nom = nom;
34    }
35
36    // Version 3 : modifier la quantité et le prix
37    void modifier(int quantite, double prix) {
38        this.quantite = quantite;
39        this.prix = prix;
40    }
41
42    // Version 4 : modifier la quantité, le prix et le nom
43    void modifier(int quantite, double prix, String nom) {
44        this.quantite = quantite;
45        this.prix = prix;
46        this.nom = nom;
47    }
48
49    // --- Méthode d'affichage ---
50    void afficher() {
51        System.out.println("Code : " + code);
52        System.out.println("Nom : " + nom);
53        System.out.println("Quantité : " + quantite);
54        System.out.println("Prix : " + prix);
55        System.out.println("-----");
```

```

54     }
55 }

```

Code 2.17 – Classe Produit avec surcharge de constructeurs et de méthodes

```

1  public class Program {
2      public static void main(String[] args) {
3
4          // Utilisation du constructeur par défaut
5          Produit p1 = new Produit();
6          p1.afficher();
7
8          // Utilisation du constructeur avec paramètres
9          Produit p2 = new Produit(123, "Galaxy S44", 50, 4500.5);
10         p2.afficher();
11
12         // Appel des différentes versions surchargées de la méthode modifier
13         p2.modifier(30); // modifie la
14         // quantité
15         p2.modifier("Galaxy S45"); // modifie le nom
16         p2.modifier(40, 4700.0); // modifie quantité
17         // et prix
18         p2.modifier(60, 4800.0, "Galaxy S46"); // modifie quantité
19         // , prix et nom
20
21         // Affichage final après modifications
22         System.out.println("Après modification :");
23         p2.afficher();
24     }
25 }

```

Code 2.18 – Classe Program illustrant l'utilisation de la surcharge

Dans cet exemple :

- La classe `Produit` illustre la **surcharge des constructeurs** et des **méthodes** ;
- La classe `Program` montre comment ces différentes formes de surcharge peuvent être utilisées dans un programme ;
- Les appels successifs à la méthode `modifier` démontrent qu'en Java, le compilateur choisit automatiquement la bonne méthode selon le nombre et le type des arguments fournis.

Conclusion

Dans cet exemple, la surcharge de constructeurs et de méthodes illustre comment une même opération (créer ou modifier un produit) peut être adaptée à différentes situations selon les informations disponibles. Cela permet de :

- simplifier la compréhension du code ;
- améliorer la maintenabilité ;
- éviter la duplication de noms de méthodes ;
- conserver une logique uniforme dans la classe.

2.9 Exercices

Cette série d'exercices a pour objectif de consolider la compréhension du concept de **surcharge** à travers deux contextes : la **surcharge des constructeurs** et la **surcharge des méthodes**.

Exercice 1 : La classe `Point`

Reprend le code source précédent de la classe `Point`.

On souhaite modéliser un point dans un plan cartésien à l'aide d'une classe `Point` possédant deux attributs : `x` et `y`.

Travail demandé :

1. Créer plusieurs constructeurs permettant d'initialiser un point selon différentes situations (par exemple, sans coordonnées, avec une seule coordonnée, ou avec les deux).
2. Proposer une méthode surchargée `deplacer()` permettant de déplacer un point :
 - soit selon une seule direction (axe des abscisses ou des ordonnées) ;
 - soit selon les deux directions simultanément.
3. Discuter l'intérêt de cette approche dans un contexte où la classe `Point` serait utilisée dans une application graphique.

Exercice 2 : La classe `Salarie`

Reprendre le code source précédent de la classe `Salarie`.

On souhaite modéliser un salarié à l'aide d'une classe `Salarie` comportant les attributs suivants : `matricule`, `nom` et `prénom`.

Travail demandé :

1. Créer plusieurs constructeurs pour permettre différentes façons d'instancier un salarié :
 - sans informations initiales ;
 - avec uniquement le nom et le prénom ;
 - avec le matricule, le nom et le prénom.
2. Créer une méthode surchargée `modifier()` permettant :
 - de modifier uniquement le nom et le prénom d'un salarié ;
 - de modifier le matricule en plus du nom et du prénom.
3. Expliquer comment le compilateur choisit la méthode appropriée lors de l'appel de `modifier()`.

Synthèse : À travers ces exercices, on constate que la surcharge (overloading) permet d'adapter le comportement d'une classe à différents contextes d'utilisation, tout en conservant la même sémantique associée au nom d'une méthode ou d'un constructeur. Il s'agit d'une forme de polymorphisme, appelée polymorphisme statique ou polymorphisme à la compilation.

Plus précisément, ce lien avec le polymorphisme peut être expliqué à travers une résolution à la compilation :

Avec la surcharge, le compilateur choisit automatiquement la version appropriée de la méthode en fonction des paramètres fournis (types, nombre, ordre). Ce choix se fait avant l'exécution, d'où le terme polymorphisme statique. Plusieurs méthodes portent le même nom mais représentent des « formes » différentes du même comportement.

Ainsi, la surcharge illustre une catégorie du polymorphisme : la capacité d'un même nom à représenter plusieurs comportements distincts selon les situations.

Chapitre 3

Encapsulation en Programmation Orientée Objet (POO)

Définition

L'encapsulation est l'un des principes fondamentaux de la programmation orientée objet (POO). Elle consiste à protéger les données internes d'une classe en limitant leur accès direct depuis l'extérieur. Autrement dit, elle permet de masquer les détails d'implémentation et de contrôler la manière dont les attributs sont lus ou modifiés.

Objectifs de l'encapsulation

- Protéger les données contre des modifications non contrôlées.
- Assurer la cohérence et la validité des objets (en imposant des règles de gestion).
- Rendre le code plus robuste et plus facile à maintenir.
- Contrôler l'accès aux informations via des méthodes spécifiques (souvent appelées **getters** et **setters**).

Principe clé

Les attributs d'une classe sont généralement déclarés avec le modificateur d'accès **private**, ce qui les rend inaccessibles directement depuis le code extérieur. Les opérations sur ces attributs se font uniquement à travers des méthodes publiques qui imposent des règles de validation.

Les méthodes d'une classe sont généralement déclarés avec le modificateur d'accès **public**. Si non, les objets de la classe ne peuvent jamais communiquer leurs structures à l'extérieur.

3.1 Mise en oeuvre de l'encapsulation

Déclaration des attributs Dans cet exemple, la classe **Produit** illustre le principe d'encapsulation :

```
1 class Produit {  
2     int code;  
3     String nom;  
4     private int qte;  
5     private double prix;  
6 }
```

Code 3.1 – Déclaration des attributs avec encapsulation

Analyse :

- `code` et `nom` sont accessibles depuis l'extérieur (non encapsulés).

- `qte` et `prix` sont déclarés **private** donc protégés contre un accès direct depuis l'extérieur : ils ne peuvent pas être modifiés directement depuis la méthode `main` ou toute autre méthode externe de la classe `Produit`.

Ajouter des méthodes avec une visibilité **public** et ce pour une alternative qui intègre le contrôle d'accès. **Protection et contrôle via des méthodes**

```

1 public void modifierPrix(double prix) {
2     if (prix > 0)
3         this.prix = prix;
4 }
5
6 public void modifierQte(int qte) {
7     if (qte > 0)
8         this.qte = qte;
9 }

```

Code 3.2 – Méthodes pour modifier les attributs protégés

la figure suivante montre comment les attributs quantité et prix sont protégés contre un accès direct depuis l'extérieur (main de Program).

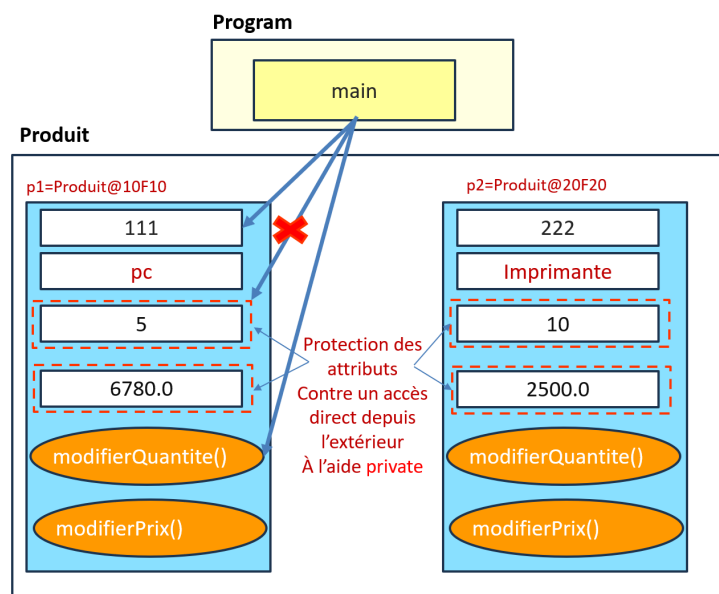


FIGURE 3.1 – Encapsulation : protection des attributs

Analyse : Les méthodes `modifierQuantite(int)` et `modifierPrix(double)` imposent des règles de gestion : - Le prix d'un produit ne peut être modifié que s'il est supérieur à zéro. - La quantité ne peut être modifiée que si elle est positive.

Ainsi, même si un code externe (main de la classe Program) donne la possibilité à l'utilisateur de saisir des valeurs incorrectes, les données ne seront pas acceptées par les méthodes `modifierQuantite(quantite)` et `modifierPrix(prix)` et ainsi les données restent cohérentes.

Illustration dans le `main()`

```

1 Produit p1 = new Produit();

```

```

2 // Accès interdit directement aux attributs privés
3 p1.qte = -100; // Erreur de compilation
4 p1.prix = -3450; // Erreur de compilation
5
6 // Accès via méthodes public
7 p1.modifierPrix(-3450); // Appel Autorisé, mais la valeur est
8   contr lée
9 p1.modifierQte(-100); //Appel Autorisé, mais la valeur est
   contr lée

```

Code 3.3 – Exemple d'utilisation dans le main

Analyse : - L'encapsulation protège l'objet contre des données incohérentes. - L'accès direct depuis l'extérieur aux attributs privés est impossible. - Le contrôle passe par les méthodes qui filtrent les valeurs incorrectes.

Méthodes membres & attributs privés

```

1 public void print() {
2     System.out.println("Qte:" + qte);
3     System.out.println("Prix:" + prix);
4 }

```

Code 3.4 – Méthode membres & attributs privés

Analyse : Même si `qte` et `prix` sont privés, ils restent accessibles à l'intérieur de la classe pour n'importe quelle méthode.

Cela montre que la visibilité `private` bloque uniquement l'accès extérieur, pas interne.

3.2 Rôle de l'encapsulation

Aspect	Description	Effet / Bénéfice
Protection des données	<code>qte</code> et <code>prix</code> sont privés	Empêche toute modification contrôlée
Validation des entrées	Méthodes <code>modifierPrix()</code> et <code>modifierQte()</code> imposent des conditions	Garantit la cohérence (valeurs positives)
Masquage de l'implémentation	L'utilisateur ne sait pas comment <code>prix</code> est stocké ou validé	Simplifie l'utilisation de la classe
Sécurité logique	L'accès se fait uniquement via des méthodes de contrôle	Réduit le risque d'erreurs d'anomalies

Conclusion

L'encapsulation, illustrée ici à travers la classe `Produit`, joue un rôle essentiel dans la fiabilité et la cohérence des objets. En rendant certains attributs `private` et en fournissant des méthodes de modification encadrées par des règles de gestion, le code devient :

- plus sécurisé,
- plus robuste,
- plus facile à maintenir : le code source mettant en oeuvre les règles doit se trouver dans un seul emplacement

Ce principe est un pilier de la POO, car il protège l'état interne des objets et garantit leur cohérence tout au long de leur cycle de vie.

3.3 Encapsulation & Getters et Setters en Java

3.3.1 Pourquoi utiliser les getters et setters ?

Limitation de l'accès direct aux données

Lorsqu'un attribut est déclaré `private`, il devient inaccessible directement depuis l'extérieur de la classe. Cependant, il faut tout de même pouvoir lire ou modifier ces données d'une manière contrôlée et sécurisée mais d'une manière qui respecte les standards et les conventions. C'est précisément le rôle des **getters** et **setters**.

Définition

Type	Rôle	Syntaxe standard (Java)
Getter	Permet de consulter la valeur d'un attribut privé	<code>public Type getNomAttribut()</code>
Setter	Permet de modifier la valeur d'un attribut privé de manière contrôlée	<code>public void setNomAttribut(Type valeur)</code>

Exemple standard

```

1 public class Produit {
2     private double prix; // attribut encapsulé
3
4     public double getPrix() {
5         return prix; // lecture du prix
6     }
7
8     public void setPrix(double prix) {
9         if (prix > 0) // règle de gestion
10            this.prix = prix;
11        else
12            System.out.println("Error dans prix!"); // vaud mieux
13                lever une exception. cf Gestion des exceptions
14    }
15 }
```

Code 3.5 – Exemple de getter et setter pour l'attribut prix

Pourquoi ils sont indispensables

- **Sécurité des données** : Empêche l'accès direct aux variables internes tout en permettant un contrôle sur leur modification.
- **Validation des valeurs** : Permet d'ajouter des règles (ex. refuser un prix négatif).
- **Lisibilité et standardisation** : `getX()` et `setX()` sont des conventions reconnues par tous les développeurs Java.

- **Compatibilité avec les frameworks** : De nombreux frameworks Java (Spring, Hibernate, JavaBeans) s'appuient sur les noms standards `get` et `set`.
- **Évolutivité** : On peut modifier la logique interne d'un getter ou setter sans impacter le code externe.

3.3.2 Application sur la classe Produit

```
1  class Produit {
2      private int code;
3      private String nom;
4      private int qte;
5      private double prix;
6
7      // Constructeur
8      public Produit(int code, String nom, int qte, double prix) {
9          this.code = code;
10         this.nom = nom;
11         this.qte = qte;
12         this.prix = prix;
13     }
14
15     // ===== GETTERS =====
16     public int getCode() { return code; }
17     public String getNom() { return nom; }
18     public int getQte() { return qte; }
19     public double getPrix() { return prix; }
20
21     // ===== SETTERS =====
22     public void setCode(int code) { this.code = code; }
23     public void setNom(String nom) { this.nom = nom; }
24     public void setQte(int qte) { if (qte > 0) this.qte = qte; }
25     public void setPrix(double prix) { if (prix > 0) this.prix =
26         prix; }
27
28     // Méthode d'affichage
29     public void print() {
30         System.out.println("Code: " + code);
31         System.out.println("Nom: " + nom);
32         System.out.println("Qte: " + qte);
33         System.out.println("Prix: " + prix);
34     }
35 }
```

Code 3.6 – Classe Produit avec getters et setters

3.3.3 Exemple d'utilisation dans le main() de la classe Program

```
1  public class Program {
2      public static void main(String[] args) {
3          Produit p1 = new Produit(123, "Galaxy S22", 50, 3450);
```

```

4      // Accès direct interdit :
5      // p1.prix = -2000;   Erreur : prix est private
6
7
8      // Accès via setters :
9      p1.setPrix(-2000); // refusé car prix < 0
10     p1.setPrix(4000);  // accepté
11
12     // Lecture via getters :
13     System.out.println("Prix actuel: " + p1.getPrix());
14
15     p1.print();
16 }
17 }

```

Code 3.7 – Utilisation sécurisée des getters et setters

3.3.4 Analyse du code source

Aspect	Rôle	Effet sur la classe
Encapsulation	Les attributs sont private	Empêche les accès directs et pose un passage par des méthodes contrôlées
Getters	Permettent la lecture sans modifier	Maintiennent la sécurité tout en donnant la visibilité nécessaire
Setters	Permettent la modification avec validation	Garantissent la cohérence et la validité des données
Standardisation	Noms get/set suivis du nom de l'attribut	Conformes à la convention Java Bean
Séparation des responsabilités	Les règles de gestion sont localisées dans la classe	Le reste du programme n'a pas à connaître la logique interne

Conclusion

L'utilisation des getters et setters est la forme la plus courante et standardisée pour opérer l'encapsulation en Java. Grâce à eux :

- Les données internes restent protégées (**private**),
- Leur lecture et écriture passent par des points de contrôle,
- Le code reste robuste, cohérent et maintenable,
- La classe respecte les standards JavaBean, facilitant l'intégration avec frameworks et outils modernes.

En résumé, les getters et setters sont la porte d'entrée officielle vers les données encapsulées d'un objet, assurant un accès maîtrisé, sécurisé et standardisé tout en favorisant une maintenance de code source.

3.3.5 La méthode `toString()`

1. Qu'est-ce que la méthode `toString()` ?

En Java, la méthode `toString()` est une méthode héritée de la classe `Object`. Elle retourne une représentation textuelle d'un objet. Par défaut, elle affiche le nom de la

classe suivi d'un identifiant mémoire, ce qui n'est généralement pas utile pour l'affichage des données d'un objet.

2. Pourquoi utiliser `toString()` ?

La redéfinition (*override*) de la méthode `toString()` permet :

- d'obtenir une description lisible et personnalisée d'un objet ;
- de faciliter l'affichage dans les instructions `System.out.println()` ;
- d'améliorer le débogage et la traçabilité des données ;
- de simplifier les tests et les impressions d'objets.

Grâce à cette méthode, il devient possible d'afficher directement un objet sans être obligé de définir une autre méthode spécifique qui affiche le contenu d'un produit comme `printDetailPProduit()`.

3. Structure complète d'une entité Java

Une classe représentant une entité métier (telle que `Produit` ou `Salarie`) doit généralement posséder :

- des attributs déclarés en `private` (principe d'encapsulation) ;
- un ou plusieurs constructeurs pour initialiser l'objet ;
- des `getters` et `setters` pour accéder aux attributs en toute sécurité ;
- éventuellement la méthode `toString()` pour obtenir une représentation claire de l'objet.

Cette structure assure un bon niveau d'encapsulation tout en facilitant l'utilisation de la classe dans un programme.

3.3.6 Exemple de mise en œuvre : intégration de `toString()` dans la classe `Produit`

```
1 class Produit {
2     private int code;
3     private String nom;
4     private int qte;
5     private double prix;
6
7     // Constructeur
8     public Produit(int code, String nom, int qte, double prix) {
9         this.code = code;
10        this.nom = nom;
11        this.qte = qte;
12        this.prix = prix;
13    }
14
15    // ===== GETTERS =====
16    public int getCode() { return code; }
17    public String getNom() { return nom; }
18    public int getQte() { return qte; }
19    public double getPrix() { return prix; }
20
21    // ===== SETTERS =====
```

```

22     public void setCode(int code) { this.code = code; }
23     public void setNom(String nom) { this.nom = nom; }
24     public void setQte(int qte) { if (qte >= 0) this.qte = qte; }
25     public void setPrix(double prix) { if (prix >= 0) this.prix =
        prix; }
26
27     // Redéfinition de toString()
28     @Override
29     public String toString() {
30         return "Produit { " +
31             "code=" + code +
32             ", nom='" + nom + '\',' +
33             ", qte=" + qte +
34             ", prix=" + prix +
35             " }";
36     }
37 }

```

Code 3.8 – Ajout de la méthode toString() dans la classe Produit

3.3.7 Exemple d'utilisation dans la classe Program

```

1  public class Program {
2      public static void main(String[] args) {
3
4          Produit p1 = new Produit(123, "Galaxy S22", 50, 3450);
5
6          // Affichage direct de l'objet
7          System.out.println(p1);
8
9          // Résultat obtenu :
10         // Produit { code=123, nom='Galaxy S22', qte=50, prix
            =3450.0 }
11     }
12 }

```

Code 3.9 – Affichage direct d'un objet grâce à toString()

3.4 Exercices

Exercice 1 — Analyse du code source

soit le code source suivant :

Program 1

```

1  class Point {
2      private int x; private int y;
3      Point(){

```



```
4  this.x=-1;this.y=-1;
5  }
6  void deplacer(int x,int y){
7  Point p=new Point();
8  p.x+=x;p.y+=y;
9  }
10 @Override
11 public String toString() {
12 return x+" "+y;
13 }
14 }
15 public class Question3 {
16 public static void main(String[] args)
17 {
18 Point p=new Point();
19 p.deplacer(3, 4);
20 System.out.println(p);
21 }
22 }
```

Code 3.10 – Program 1

Questions :**1. Si le Program1 ne contient pas d'erreur d'exécution :**

— Résultat affiché par le programme :

2. Si le programme contient des erreurs :

— Lignes contenant des erreurs :

— Corrections à effectuer :

— Résultat à afficher après correction :

Exercice 2 — Encapsulation et contrôle des données

1. Écrire le code source d'une classe `Point` qui :
 - possède deux attributs privés : `x` et `y` de type `double` ;
 - fournit des **getters** pour accéder aux coordonnées ;
 - fournit des **setters** qui empêchent que `x` ou `y` deviennent négatifs. Si une valeur négative est fournie, elle doit être remplacée par 0 ;
 - possède un constructeur qui initialise les coordonnées avec des valeurs passées en paramètres, en respectant la règle des coordonnées non négatives. Penser à la réutilisation de code.
 - redéfinir la méthode `toString()` pour afficher le point sous la forme `(x, y)` ;
 - possède une méthode `deplacer(double dx, double dy)` qui modifie les coordonnées du point, tout en garantissant que les coordonnées restent positives. Cette méthode est surchargée avec une version qui déplace le point de manière égale sur les deux axes (`deplacer(double d)`).
 - ajouter (redéfinir) la méthode `toString()` qui retourne le détail d'un point
2. Créer une classe exécutable `Program` qui permet de :
 - Créer un point `p1` avec des coordonnées valides et afficher ses coordonnées ;
 - Tenter de créer un point `p2` avec des coordonnées négatives et vérifier que les getters renvoient des coordonnées corrigées ;
 - Déplacer `p1` en utilisant `deplacer(dx, dy)` et `deplacer(d)` avec des valeurs négatives et positives, et afficher les nouvelles coordonnées après chaque déplacement ;
 - afficher le contenu des objets en appelant directement `System.out.print(p1)`. même chose pour `p2`.
 - Vérifier que les coordonnées ne deviennent jamais négatives après les déplacements.
3. **Question** : Expliquez comment l'encapsulation (attributs privés et setters) garantit que les coordonnées restent toujours valides.

Exercice 3 — Créer une classe Salarié (avec getters/setters)

1. Écrire le code source d'une classe `Salarie` définie par les éléments suivants :
 - **matricule, nom, age, salaire** : tous les attributs doivent être `private`
 - Fournir les **getters** et **setters** pour chaque attribut afin de contrôler leur accès et modification
 - Ajouter une règle de gestion pour que le salaire doit être supérieur à 0
 - Est ce que le constructeur respecte cette règle ? Penser à réutiliser le code
 - Ajouter une règle pour que l'âge doit être compris entre 18 et 65 ans
 - Ajouter un constructeur qui permet d'initialiser tous les champs d'un salarié. Penser à la réutilisation de code afin de prendre en compte ces deux règles
 - ajouter (redéfinir) la méthode `toString()` qui retourne le détail d'un salarié
2. Créer une classe exécutable `Program` qui permet de :
 - Créer un premier salarié `s1` via le constructeur
 - Modifier ses attributs via les setters

- Afficher ses informations via les getters
 - Afficher ses informations via la méthode toString()
 - Tester les validations : tenter d'affecter un âge négatif ou un salaire nul
 - Créer un deuxième salarié `s2` en utilisant le constructeur avec paramètres et afficher ses informations. Essayer de vérifier s'il accepte un salaire ou age qui contredit les règles de gestion.
3. **Question** : Expliquez comment les getters et setters garantissent la cohérence et la sécurité des données du salarié.

Chapitre 4

Introduction aux Collections en Java

4.1 Présentation générale des collections

En Java, une **collection** est un objet qui regroupe plusieurs éléments (objets ou types primitifs via les classes enveloppes). Les collections permettent de stocker, gérer et manipuler un ensemble d'éléments de manière dynamique. Elles remplacent souvent les tableaux lorsque la taille de la structure n'est pas fixe ou que l'on a besoin de méthodes avancées pour ajouter, supprimer, trier ou rechercher des éléments.

Il existe plusieurs interfaces (les interfaces ne sont pas traitées dans ce document). et classes pour gérer les collections en Java : **List**, **Set**, **Queue**, **Map**, chacune ayant des implémentations spécifiques adaptées aux besoins.

Avantages des collections :

- Gestion dynamique de la taille
- Méthodes intégrées pour ajouter, supprimer, trier, rechercher
- Meilleure lisibilité et maintenabilité du code

4.2 List et son implémentation ArrayList

Une **List** est une collection qui permet de stocker des éléments. Chaque élément possède un **indice** qui commence à 0. L'implémentation la plus courante est **ArrayList**, qui offre des performances efficaces pour l'accès par indice et la modification des éléments.

il est à noter que l'implémentation par défaut de **ArrayList** autorise les doublants.

Pour éviter les doublons dans une **ArrayList**, il revient au développeur de contrôler les insertions, par exemple en vérifiant si l'élément existe déjà avant de l'ajouter.

Déclaration et instanciation :

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 List<Integer> nombres = new ArrayList<>();
```

ArrayList fournit des méthodes pratiques comme :

- **add(element)** : ajouter un élément
- **get(index)** : récupérer un élément
- **remove(index)** : supprimer un élément
- **size()** : connaître le nombre d'éléments
- **contains(element)** : tester la présence d'un élément

4.3 Exemple avec des entiers

le code suivant montre comment créer et ajouter des éléments(entiers) à une liste.

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class Program {
5      public static void main(String[] args) {
6          List<Integer> nombres = new ArrayList<>();
7          nombres.add(33);
8          nombres.add(44);
9          nombres.add(55);
10         nombres.add(Integer.valueOf(77)); //valueOf est une méthode
           thode static qui renvoie un objet de type Integer
11
12
13         System.out.println("Liste initiale: " + nombres);
14
15         // Accès par indice
16         System.out.println("Premier élément: " + nombres.get(2));
17
18         // Supprimer un élément
19         nombres.remove(1);
20         System.out.println("Après suppression: " + nombres);
21     }
22 }
```

Code 4.1 – Utilisation de ArrayList avec des entiers

la figure suivante montre les espaces mémoires concernées par le code source ci-dessus qui gère une liste d'entiers.

dans la figure, on voit bien qu'on stocke dans la liste des références des objets et non directement des valeurs entières.

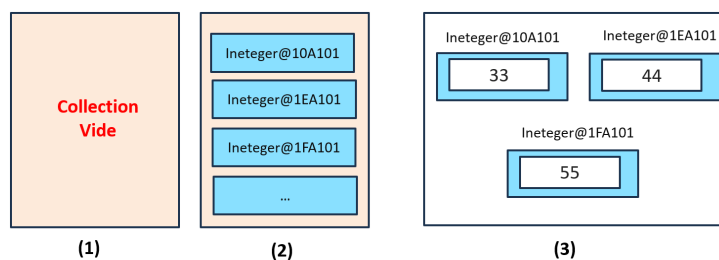


FIGURE 4.1 – Une collection d'entiers

Analyse du code source :

- Ligne 6 : Initialisation de la collection. créer un conteneur vide. (1) de la figure 4.1.

la partie List<integer> indique que la list correspond à une liste de Integer.

on ne peut pas utiliser List<int> car les collections gèrent des objets et non des variables de type primitif.

pour cette raison que chaque type primitif à un envelopper (Wrapper).

- int : le wrapper est la classe Integer
- char : le wrapper est la classe Character
- long : le wrapper est la classe Long
- double : le wrapper est la classe Double
- boolean : le wrapper est la classe Boolean
- etc

Sans new, le conteneur n'existe pas dans la mémoire.

- Ligne 7 à 9 : Ajouter des éléments de type entier à la collection. En réalité, chaque entier (primitif) est converti (casting) en un objet de type Integer. (2) de la figure 4.1 montre qu'on ajoute à la collection la référence et (3) montre le contenu de chaque référence
- Ligne 10 : Ajouter des éléments de en utilisant directement Integer.valueOf qui retourne un Integer.
- Ligne 13 : afficher le contenu de la liste
- Ligne 16 : récupérer et afficher l'élément d'indice 2
- Ligne 19: suppression d'un éléments

4.4 Exemple avec des objets Produit

On peut utiliser les ArrayList pour gérer des objets complexes, comme la classe Produit vue précédemment avec encapsulation, getters/setters et surcharge.

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class Program {
5      public static void main(String[] args) {
6          List<Produit> produits = new ArrayList<>();
7          produits.add(new Produit(101, "Galaxy S24", 20, 4500));
8          produits.add(new Produit(102, "iPhone 16", 10, 8900));
9
10         for (Produit p : produits) {
11             System.out.println(p); // toString() affiche les dé
12                                     tails
13         }
14
15         // Modifier le prix du premier produit
16         produits.get(0).setPrix(4600);
17         System.out.println("Après modification: " + produits.get
18                               (0)); // redéfinir la méthode toString dans la classe
19                                     Produit pour un affichage adéquat
20         for (Produit p:produis)
21             System.out.println(p);
22     }
23 }
```

Code 4.2 – Gestion d'une collection de produits

la figure suivante montre les ressources mémoires allouées à la fin de l'exécution de la classe Program :

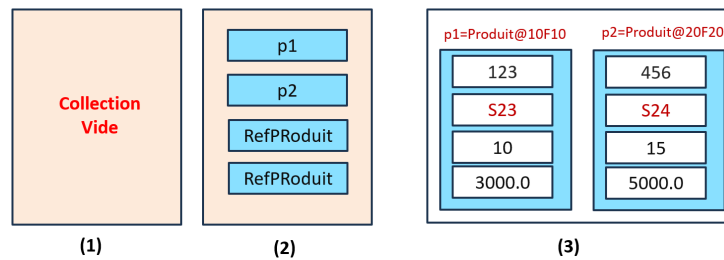


FIGURE 4.2 – Une collection de produits

Analyse du code source :

- Ligne 6) : initialisation de la liste
- Ligne 7 à 8 : Ajouter des éléments à la liste
- Ligne 12 à 12 : afficher le contenu de la liste à l'aide d'une nouvelle forme des boucles qui correspond au parcours des collections (java7).
- Ligne 15 : récupérer la référence (d'un Produit) qui se trouve à l'indice 0 puis changer le prix cette référence
- Ligne 16 : afficher le contenu d'un produit. Si la méthode toString n'est pas définie dans la classe Produit, le programme se limite à afficher la référence.
- Ligne 17 à 18 : afficher le contenu de toute la liste. Si la méthode toString n'est pas définie dans la classe Produit, le programme se limite à afficher les références.

4.5 Classe utilitaire Collections

Présentation de Collections

La classe utilitaire `Collections` (du package `java.util`) fournit un ensemble de méthodes static permettant de manipuler des collections Java, notamment :

- trier une liste : `Collections.sort()`,
- mélanger les éléments : `Collections.shuffle()`,
- rechercher une valeur : `Collections.binarySearch()`,
- obtenir les valeurs extrêmes : `Collections.max()`, `Collections.min()`,
- remplir une liste : `Collections.fill()`,
- copier des collections, etc.

Exemple 1 — Trier des types primitifs

La méthode `Collections.sort()` permet de trier directement une liste de valeurs simples (entiers, chaînes de caractères, etc.). Java applique alors l'ordre naturel (*natural ordering*) défini par les types correspondants.

```

1 List<Integer> nombres = Arrays.asList(12, 5, 30, 2, 18);
2 Collections.sort(nombres);
3 System.out.println("Liste triée : " + nombres);
4 // Affiche : [2, 5, 12, 18, 30]
```

Code 4.3 – Tri d'une liste de types primitifs

Cet exemple montre qu'il suffit d'une seule ligne pour trier une liste, car les entiers possèdent déjà un ordre naturel, défini dans la classe `Integer` via l'interface `Comparable`.

Exemple 2 — Trier des objets

Lorsqu'il s'agit d'objets, Java ne sait pas par défaut comment les comparer. Il faut donc fournir un `Comparator` qui décrit la règle de tri.

Pourquoi ajouter un `Comparator` ? Lorsque l'on manipule une liste d'objets de type `Produit`, Java ne sait pas automatiquement comment comparer deux produits entre eux. En effet, la classe `Produit` ne définit aucun ordre naturel, car elle n'implémente pas l'interface `Comparable`.

Ainsi, la méthode `Collections.sort()` ne peut pas trier une liste de produits sans que l'on précise explicitement la règle de comparaison.

Pour cette raison, il est nécessaire d'ajouter un `Comparator` qui indique à Java selon quel critère effectuer le tri (ici, le tri par nom). Le code suivant fournit cette règle de tri :

Exemple : trier des produits par nom

```
1 import java.util.*;
2
3 class Produit {
4     private int code;
5     private String nom;
6     private int quantite;
7     private double prix;
8
9     public Produit(int code, String nom, int quantite, double
10         prix) {
11         this.code = code;
12         this.nom = nom;
13         this.quantite = quantite;
14         this.prix = prix;
15     }
16
17     public int getCode() { return code; }
18     public String getNom() { return nom; }
19     public int getQuantite() { return quantite; }
20     public double getPrix() { return prix; }
21
22     @Override
23     public String toString() {
24         return code + " - " + nom + " (" + quantite + " pcs, " +
25             prix + " DH)";
26     }
27 }
28
29 // Comparator pour trier les produits par nom
30 class ProduitComparator implements Comparator<Produit> {
31     @Override
```

```
30     public int compare(Produit p1, Produit p2) {
31         return p1.getNom().compareTo(p2.getNom());
32     }
33 }
34
35 public class Program {
36     public static void main(String[] args) {
37
38         List<Produit> produits = new ArrayList<Produit>();
39         produits.add(new Produit(101, "Stylo", 50, 3.5));
40         produits.add(new Produit(102, "Cahier", 120, 7.0));
41         produits.add(new Produit(103, "Crayon", 200, 2.0));
42         produits.add(new Produit(104, "Gomme", 80, 1.5));
43
44         // Tri selon notre propre Comparator
45         Collections.sort(produits, new ProduitComparator());
46
47         System.out.println("Produits triés :");
48         for (Produit p : produits) {
49             System.out.println(p);
50         }
51     }
52 }
```

Code 4.4 – Tri d’objets avec un Comparator

Analyse

- La classe `Produit` représente un objet métier contenant quatre attributs : `code`, `nom`, `quantite` et `prix`.
- Java ne peut pas trier automatiquement une liste de produits, car il ne connaît pas la règle de comparaison.
- La classe `ProduitComparator` fournit cette règle : comparer les produits selon leur `nom`.
- La méthode `Collections.sort()` utilise alors ce comparator pour trier la liste.

Modification proposée

Pour trier les produits par prix plutôt que par nom, il suffit de modifier le comparator :

```
1 class ProduitPrixComparator implements Comparator<Produit> {
2     @Override
3     public int compare(Produit p1, Produit p2) {
4         return Double.compare(p1.getPrix(), p2.getPrix());
5     }
6 }
```

Code 4.5 – Tri des produits par prix

Questions

1. **Analyse du code source** : Expliquez le rôle de chaque classe, l'utilisation de `Comparator`, ainsi que le fonctionnement de la méthode `Collections.sort`.
2. **Modification à apporter** : Modifiez le programme de manière à trier les produits **par prix** au lieu de les trier par nom.

4.6 Exercices

Exercice 1 — Gestion des salariés

1. Créer une classe `Salarie` avec encapsulation, getters et setters, et règles de gestion pour l'âge et le salaire.
créer une classe `Program` ensuite,
2. Créer une `ArrayList<Salarie>` et ajouter plusieurs salariés.
3. Afficher les informations de tous les salariés.
4. rechercher un salarié dans liste en utilisant le matricule
5. Modifier le salaire ou l'âge d'un salarié via ses setters et afficher les changements.

Exercice 2 — Gestion de points et calcul du plus proche point à l'origine

1. Créer une classe `Point` avec encapsulation, getters et setters. Les coordonnées ne peuvent pas être négatives, même après déplacement.
créer une classe `Program` ensuite,
2. Créer une `ArrayList<Point>` et ajouter plusieurs points.
3. Afficher tous les points.
4. Écrire une méthode pour trouver le point le plus proche de l'origine (0,0) parmi la liste.

Exercice 3 : Trier une liste de noms d'étudiants

On considère une liste contenant des chaînes de caractères représentant des noms d'étudiants.

- Déclarez une liste de type `List<String>`.
- Ajoutez-y au moins 6 noms d'étudiants (dans un ordre non trié).
- Utilisez la méthode `Collections.sort()` pour trier cette liste.
- Affichez la liste avant et après le tri.

Exercice 4 : Trier une liste de salariés

On souhaite gérer des salariés décrits par les attributs suivants :

- `matricule` (int)
- `nom` (String)
- `prenom` (String)

- salaire (double)
- telephone (String)

Travail demandé :

1. Créez une classe `Salarie` contenant les attributs précédents, un constructeur et la méthode `toString()`.
2. Créez une liste `List<Salarie>` et ajoutez-y au moins 5 salariés.
3. Implémentez un `Comparator<Salarie>` permettant de trier les salariés par **nom**.
4. Implémentez un autre `Comparator<Salarie>` permettant de trier les salariés par **salaire**.
5. Utilisez `Collections.sort()` pour effectuer le tri.
6. Affichez la liste avant et après le tri pour le tris par nom puis par salaire.

Tri et affichage :

```

1 List<Salarie> liste = new ArrayList<>();
2 liste.add(new Salarie(1, "El Idrissi", "Omar", 8000, "0600001111"
3 ));
4 liste.add(new Salarie(2, "Benali", "Sara", 7500, "0600223344"));
5 liste.add(new Salarie(3, "Khalil", "Amine", 9000, "0600556677"));
6 liste.add(new Salarie(4, "Chafiq", "Nadia", 8200, "0600778899"));
7 liste.add(new Salarie(5, "Ait Taleb", "Rachid", 7000, "0600112233"
8 ));
9
10 System.out.println("Avant tri :");
11 for (Salarie s : liste) System.out.println(s);
12
13 Collections.sort(liste, new SalarieNomComparator());
14
15 System.out.println("\nAprès tri :");
16 for (Salarie s : liste) System.out.println(s);
17
18 Collections.sort(liste, new SalarieSalaireComparator());
19
20 System.out.println("\nAprès tri :");
21 for (Salarie s : liste) System.out.println(s);

```

//_____

Exercice 3 — K-plus proches voisins (KNN)

1. Utiliser la classe `Point` pour représenter des données en 2D.
2. Créer une `ArrayList<Point>` contenant plusieurs points.
3. Écrire une méthode qui, pour un point donné, retourne les `k` points les plus proches parmi la liste.
4. Tester la méthode avec différents points et valeurs de `k`.

Chapitre 5

Conception et implémentation avancées : les relations entre les classes

Dans la programmation orientée objet, les classes ne vivent pas isolées : elles interagissent entre elles pour former un système cohérent. Ces interactions sont représentées par des **relations**, qui permettent de modéliser des liens logiques tels que la composition, l'agrégation ou la simple association.

Dans ce chapitre, nous présentons deux relations essentielles :

- la relation **OneToOne** (un objet est lié à un seul autre objet),
- la relation **ManyToOne** (plusieurs objets sont liés à un seul objet).

5.1 Relation OneToOne

5.1.1 Description générale

Une relation **OneToOne** signifie qu'un objet A contient ou utilise exactement un objet B. Cette relation est utile lorsque deux entités du système sont étroitement liées. Par exemple, un **Cercle** possède un **Point** représentant son centre. Chaque cercle a un seul centre, et ce centre est représenté par un objet de la classe **Point**.

5.1.2 Exemple : Classe Cercle et Point

Le cercle est défini par :

- un centre de type **Point**,
- un rayon,
- une couleur.

la figure ci dessous montre le diagramme de classe exprimant cette relation.

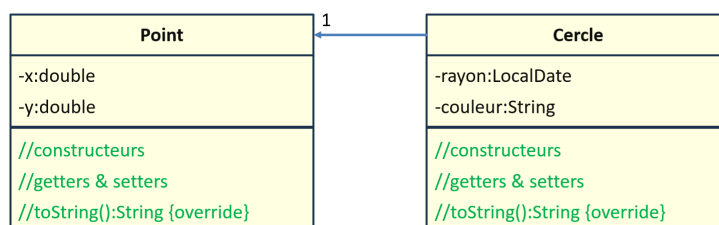


FIGURE 5.1 – Relation OneToOne

La présence d'un attribut de type **Point** exprime directement la relation OneToOne.
Code source de la classe Point :

```

1 public class Point {
2     private double x;
3     private double y;
4
5     public Point(double x, double y) {
6         this.x = x;
7         this.y = y;
8     }
9
10    public double getX() { return x; }
11    public double getY() { return y; }
12
13    public void setX(double x) { this.x = x; }
14    public void setY(double y) { this.y = y; }
15 }

```

Code 5.1 – Classe Point

Code source de la classe Cercle :

```

1 public class Cercle {
2     private Point centre;    // Relation OneToOne
3     private double rayon;
4     private String couleur;
5
6     public Cercle(Point centre, double rayon, String couleur) {
7         this.centre = centre;
8         this.rayon = rayon;
9         this.couleur = couleur;
10    }
11
12    public Point getCentre() { return centre; }
13    public double getRayon() { return rayon; }
14    public String getCouleur() { return couleur; }
15
16    public void setRayon(double rayon) { this.rayon = rayon; }
17    public void setCouleur(String couleur) { this.couleur =
18        couleur; }

```

Code 5.2 – Classe Cercle avec relation OneToOne

5.1.3 Analyse

- Le cercle **dépend** d'un objet Point pour fonctionner.
- Le constructeur impose que chaque Cercle possède un centre dès sa création.
- Le lien est direct : un seul cercle → un seul point.

5.1.4 Code source pour tester et utiliser les éléments des classes précédentes

Pour valider la relation OneToOne entre les classes `Cercle` et `Point`, nous proposons le programme suivant. Il permet de créer un point, de créer un cercle basé sur ce point, puis d'afficher ses informations.

```

1 public class ProgramOneToOne {
2     public static void main(String[] args) {
3         // Création du centre du cercle
4         Point centre = new Point(4.5, 3.2);
5
6         // Création du cercle
7         Cercle c = new Cercle(centre, 5.0, "Rouge");
8
9         // Affichage des informations
10        System.out.println("=== Informations du cercle ===");
11        System.out.println("Centre : (" + c.getCentre().getX() +
12            ", " + c.getCentre().getY() + ")");
13        System.out.println("Rayon : " + c.getRayon());
14        System.out.println("Couleur : " + c.getCouleur());
15    }
}

```

Code 5.3 – Programme de test pour la relation OneToOne

5.1.5 Analyse du programme

- Le centre est un objet `Point`, créé indépendamment.
- Le cercle reçoit cet objet en paramètre : c'est la relation OneToOne.
- Le programme illustre l'accès indirect au centre via `c.getCentre()`.

5.2 Relation ManyToOne

5.2.1 Description générale

Une relation **ManyToOne** signifie qu'un objet A peut regrouper plusieurs objets B. Autrement dit : **plusieurs éléments appartiennent au même ensemble**. C'est le cas d'un **Ticket** qui contient plusieurs **Articles**. Un seul ticket regroupe l'ensemble des achats effectués.

5.2.2 Exemple : Ticket de café et Articles

Le ticket est défini par :

- une liste d'articles,
- une date,
- un total calculé automatiquement.

la figure ci dessous montre le diagramme de classe exprimant cette relation.

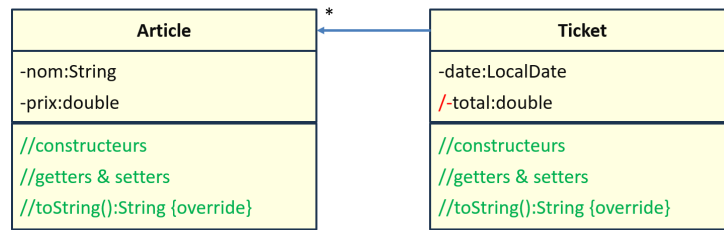


FIGURE 5.2 – Relation ManyToOne

Code source de la classe Article :

```

1 public class Article {
2     private String nom;
3     private double prix;
4
5     public Article(String nom, double prix) {
6         this.nom = nom;
7         this.prix = prix;
8     }
9
10    public String getNom() { return nom; }
11    public double getPrix() { return prix; }
12    @Override
13    public String toString(){return nom + " "+prix;}
14 }
  
```

Code 5.4 – Classe Article

Code source de la classe Ticket :

```

1 import java.util.ArrayList;
2 import java.util.Date;
3 import java.util.List;
4
5 public class Ticket {
6     private List<Article> articles = new ArrayList<>(); //
7     //ManyToOne
8     private Date date;
9     private double total;
10
11    public Ticket(Date date) {
12        this.date = date;
13    }
14
15    public void ajouterArticle(Article a) {
16        articles.add(a);
17        calculerTotal();
18    }
19
20    private void calculerTotal() {
21        total = 0;
22    }
23 }
  
```



```

21         for (Article a : articles) {
22             total += a.getPrix();
23         }
24     }
25
26     public double getTotal() { return total; }
27     public List<Article> getArticles() { return articles; }
28     public Date getDate() { return date; }
29 }

```

Code 5.5 – Classe Ticket avec relation ManyToOne

5.2.3 Analyse

- La classe `Ticket` contient plusieurs objets `Article`.
- À chaque ajout, le total est recalculé automatiquement.
- Le **Ticket** centralise les informations : plusieurs articles → un seul ticket.

5.2.4 Code source pour tester et utiliser les éléments des classes précédentes

Nous testons maintenant la relation `ManyToOne` où un `Ticket` regroupe plusieurs `Articles`. Le programme construit plusieurs articles, les ajoute au ticket puis affiche les informations finales.

```

1  import java.util.Date;
2
3  public class ProgramManyToOne {
4      public static void main(String[] args) {
5          // Création du ticket
6          Ticket ticket = new Ticket(new Date());
7
8          // Création et ajout des articles
9          Article a1 = new Article("Café", 12.00);
10         Article a2 = new Article("Croissant", 7.50);
11         Article a3 = new Article("Sandwich", 25.00);
12
13         ticket.ajouterArticle(a1);
14         ticket.ajouterArticle(a2);
15         ticket.ajouterArticle(a3);
16
17         // Affichage des articles
18         System.out.println("=== Ticket du " + ticket.getDate() +
19                             " ===");
20         for (Article a : ticket.getArticles()) {
21             System.out.println(a);
22         }
23         // Affichage du total

```

```

24         System.out.println("Total : " + ticket.getTotal() + " DH"
25         );
26     }
}

```

Code 5.6 – Programme de test pour la relation ManyToOne

5.2.5 Analyse du programme

- Chaque article est créé séparément, ce qui montre bien l'indépendance des objets.
- L'ajout dans la liste du ticket matérialise la relation ManyToOne.
- Le total est recalculé automatiquement, montrant une responsabilité interne à la classe `Ticket`.

5.3 Exercices

Exercice 1 : gestion des salariés

Énoncé

On souhaite modéliser une relation **ManyToOne** entre les classes `Salarie` et `Departement`. Un département peut regrouper plusieurs salariés.

Les deux classes `Departement` et `Salarie` sont définies de la manière suivante :

- **Departement** : `id`, `nom`
- **Salarie** : `matricule`, `nom`, `prenom`, `salaire`

Travail demandé

1. Reprendre la classe salarié du cours
2. implémenter la classe `Departement`
3. Créer une classe `Program` exécutable qui teste l'ensemble des éléments de la manière suivante :
 - Créer deux départements et plusieurs salariés, puis affecter chaque salarié à un département.
 - **Afficher le détail d'un département** : nom du département, liste des salariés et **total des salaires**.
 - **Trier la liste des salariés d'un département par salaire croissant** avant l'affichage.

Exercice 2 : Gestion des comptes bancaires

On souhaite modéliser un système simple de gestion de comptes bancaires. Un **compte bancaire** appartient à un seul client et contient un ensemble d'opérations (versements ou retraits).

Pour mieux comprendre la logique de conception orientée objet utilisée dans cet exercice, il est possible de consulter la démonstration suivante :

Démonstration (YouTube) : <https://www.youtube.com/watch?v=RkgD_bj_kJE>

Les classes à modéliser sont les suivantes :

- **Client** : cin, nom, prenom, telephone
- **Operation** : date, montant, type (“VERS” ou “RETR”)
- **Compte** : numero, dateCreation, solde, client, operations

La classe **Compte** doit contenir une méthode supplémentaire :

- double `getSolde()` : retourne le solde actuel du compte.

Travail demandé

1. Implémenter la classe **Client**.
2. Implémenter la classe **Operation**.
3. Implémenter la classe **Compte** en respectant les contraintes suivantes :
 - Une liste d’opérations doit être maintenue dans chaque compte.
 - La méthode `getSolde()` doit recalculer le solde à partir de l’ensemble des opérations :
 - “VERS” augmente le solde.
 - “RETR” diminue le solde.
4. Créer une classe **Program** exécutable contenant le scénario suivant :
 - Créer un client.
 - Créer un compte bancaire associé à ce client.
 - Ajouter plusieurs opérations (versements et retraits).
 - Afficher le détail complet du compte :
 - numéro du compte,
 - informations du client,
 - liste des opérations,
 - solde final du compte.
 - **Trier les opérations par date croissante** avant l’affichage.